03

MGU BCA - 3RD SEM

Database Management Systems

Module 3 - Normalization and Transaction Management

By Aben Sojan

Exam - 6 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

TOPIC 1

Anomalies in Database Design

1

An anomaly is a problem caused by storing the same fact in many places in an unnormalised table.

Three types: insertion anomaly (cannot insert), deletion anomaly (lose data on delete), modification anomaly (must update many rows).

Anomalies are the main reason normalization exists.

QUESTION BANK

1 What is modification anomaly? (Dec 2021, Mar 2021)

SURE MARK

Modification (update) anomaly: changing one fact forces changes in many rows, risking inconsistency.

It happens when the same fact is repeated because the table is not normalized.

Example: in Enroll(rollno, name, course), the student name Aben appears in 5 rows; correcting the spelling needs 5 updates.

If even one row is missed, the database holds two different names for the same student - inconsistent data.

Normalization removes the anomaly by storing each fact once: Student(rollno, name) plus Enroll(rollno, course).

This is why anomalies are the main motivation for normalization.

2 What is insertion anomaly? (Oct 2019)

IMPORTANT

Insertion anomaly: you cannot insert a fact because another unrelated fact is missing.

Example: in Dept_Emp(deptno, deptname, empno, empname), you cannot add a new department until it has at least one employee.

The department's existence wrongly depends on having employees.

After normalization into Department(deptno, deptname) and Employee(empno, empname, deptno), departments can be added freely.

This anomaly also wastes design freedom: the table structure forces a business rule that should not exist.

In short, insertion anomaly means missing data in one column blocks perfectly valid data in another.

3 What is deletion anomaly?

IMPORTANT

Deletion anomaly: deleting one fact accidentally deletes another unrelated fact.

Example: in Dept_Emp(deptno, deptname, empno, empname), deleting the last employee of a department wipes out the department's name too.

The department information is lost just because its employees left.

Normalization fixes it: deleting from Employee never touches the Department table.

It is the mirror of the insertion anomaly: two independent facts share one row, so removing one destroys the other.

This is why each entity type gets its own table during normalization.

TOPIC 2

Functional Dependencies

2

A functional dependency X -> Y means the value of X decides the value of Y.

Example: rollno -> name, because one roll number has exactly one name.

Trivial FD: Y is a part of X, e.g. {rollno, name} -> name.

Non-trivial FD: Y is not a part of X, e.g. rollno -> name.

Transitive FD: X -> Y and Y -> Z gives X -> Z indirectly.

4 What is a functional dependency? Explain its types. (Oct 2019)

IMPORTANT

A functional dependency X -> Y means that in a relation, each value of the attribute set X determines exactly one value of the attribute set Y.

It is a constraint on the data: if two rows agree on X, they must also agree on Y. Example: rollno -> name, because one roll number belongs to exactly one student.

FDs come in three types:

1. Trivial FD: Y is a subset of X, so it holds automatically. Example: {rollno, name} -> rollno.

2. Non-trivial FD: Y is not a subset of X, expressing a real rule of the data. Example: rollno -> name.

3. Transitive FD: X -> Y and Y -> Z hold, so X -> Z holds indirectly. Example: rollno -> dept and dept -> hod give rollno -> hod.

FDs are the raw material of normalization: an FD whose left side is not a key signals redundancy that a normal form will remove.

They also identify keys: if X determines every attribute of the relation, X is a super key.

5 Give two examples of functional dependencies in a Student table.

IMPORTANT

Example 1: rollno -> name. One roll number belongs to exactly one student, so knowing the roll number fixes the name.

Example 2: rollno -> dept. One roll number belongs to exactly one department, so knowing the roll number fixes the department.

Both are non-trivial FDs because the right side is not part of the left side; they capture real rules of the college data.

A non-example: name -> rollno is false, because two students can share the same name, so one name does not fix one roll number.

Another valid FD is {rollno, subject} -> marks: one student in one subject has exactly one mark.

Since rollno determines every column of the Student table, rollno is a super key, in fact the primary key.

In an exam, always test a claimed FD by asking: can one left-side value ever map to two different right-side values? If yes, it is not an FD.

These FDs are what normalization later uses to detect partial and transitive dependencies.

6 **Discuss the role of functional dependencies in determining keys, superkeys and candidate keys. (Model paper)**

IMPORTANT

A functional dependency X -> Y means X's value fixes Y's value; this is how keys are discovered.

A superkey is any attribute set whose closure includes all attributes of the table.

A candidate key is a minimal superkey - no smaller subset still determines everything.

Example: in Student(rollno, name, dept) with rollno -> name and rollno -> dept, the closure of {rollno} is all attributes, so rollno is a candidate key (and the primary key).

Armstrong's axioms and attribute closure are the mechanical tools used to test these.

So FDs are the foundation: keys are simply the minimal attribute sets that functionally determine the whole row.

TOPIC 3

Armstrong's Axioms

3

Reflexivity: if Y is a subset of X, then X -> Y.

Augmentation: if X -> Y, then XZ -> YZ for any Z.

Transitivity: if X -> Y and Y -> Z, then X -> Z.

These three rules derive every other valid functional dependency.

QUESTION BANK

7 **State and explain Armstrong's axioms.**

IMPORTANT

Armstrong's axioms are three inference rules that derive every FD logically implied by a given set of FDs.

1. Reflexivity: if Y is a subset of X, then X -> Y. Example: {rollno, name} -> rollno, since rollno is already inside the left side.

2. Augmentation: if X -> Y, then XZ -> YZ for any attribute set Z. Example: from rollno -> name we get {rollno, dept} -> {name, dept}.

3. Transitivity: if X -> Y and Y -> Z, then X -> Z. Example: rollno -> dept and dept -> hod give rollno -> hod.

The axioms are sound: they derive only FDs that truly hold. They are also complete: every FD that logically follows can be derived with them.

Useful derived rules include union (X -> Y and X -> Z give X -> YZ) and decomposition (X -> YZ gives X -> Y and X -> Z).

They are the engine behind closure computation: repeatedly applying the axioms to an attribute set yields everything it determines.

That is why closure, keys and normal-form tests all rest on these three simple rules.

8 **Using Armstrong axioms, show that if A -> B and B -> C then A -> C.**

IMPORTANT

Given: A -> B and B -> C hold in the relation.

The transitivity axiom states: if X -> Y and Y -> Z, then X -> Z.

Match X = A, Y = B, Z = C against the axiom's pattern.

Our two given FDs fit the pattern exactly: A -> B is X -> Y, and B -> C is Y -> Z.

Applying the axiom therefore yields A -> C.

This holds in every relation instance satisfying the given FDs, because the axiom is sound.

It shows why dependency chains collapse: a two-step determination is equivalent to a direct one.

Closure computation is exactly this proof repeated: from A we collect B, then from B we collect C.

TOPIC 4

Closure of Functional Dependencies

4

The closure of X, written X+, is every attribute that X can determine.

It is found by repeatedly applying FDs until nothing new is added.

Closure is used to find candidate keys and to test normal forms.

Example: with rollno -> dept and dept -> hod, closure of rollno is {rollno, dept, hod}.

QUESTION BANK

9 What is attribute closure? How is it used?

IMPORTANT

The attribute closure of X, written X+, is the complete set of attributes functionally determined by X under the given FDs.

Computation method: start with the set containing X, then repeatedly find any FD whose left side is already inside the set and add its right side, stopping when nothing new can be added.

Example: with rollno -> dept and dept -> hod, the closure of {rollno} is {rollno, dept, hod}.

Use 1, finding keys: if X+ contains every attribute of the relation, then X is a super key.

Use 2, testing normal forms: we need X+ to know whether an attribute is prime (part of some key) and whether a FD's left side is a key.

Use 3, checking implication: the FD X -> Y is implied exactly when Y is inside X+.

Closure is therefore the working tool of normalization: it turns a small FD set into everything those FDs imply.

In the exam, always show each step of the closure build-up, since the working earns the marks.

10 Find the closure of {rollno} given rollno -> dept and dept -> hod.

IMPORTANT

Given FDs: rollno -> dept and dept -> hod. We build the closure of {rollno} step by step.

Step 1: start with the initial set {rollno}.

Step 2: the FD rollno -> dept has its left side inside the set, so add dept. The set becomes {rollno, dept}.

Step 3: the FD dept -> hod now applies because dept is in the set, so add hod. The set becomes {rollno, dept, hod}.

Step 4: check again; no FD can add anything new, so the process stops.

Therefore the closure of {rollno} is {rollno, dept, hod}.

This also proves the transitive FD rollno -> hod, since hod ended up inside the closure.

Note: if the relation is Student(rollno, name, dept, hod), this closure does not include name, so {rollno} alone is not a key of that full relation.

TOPIC 5

First Normal Form (1NF)

5

1NF: every cell holds a single atomic value; no repeating groups or lists in one cell.

Example violation: one row storing two phone numbers in a single phone column.

Fix: split into separate rows, one phone number per row.

1NF is the entry ticket: every higher normal form needs 1NF first.

QUESTION BANK

11 **What is 1NF? Convert an unnormalised table to 1NF with an example. (Dec 2021)**

IMPORTANT

First Normal Form requires that every cell of a table holds a single atomic value: no lists, no repeating groups, no multi-valued attributes packed into one column.

Example of violation: Student(rollno, name, phones) where one row stores phones = '9876543210, 9123456780' in a single cell.

This breaks queries: asking 'which student has phone 9123456780' needs string searching instead of a clean equality test.

Conversion to 1NF: split the multi-valued cell into separate rows, one value per row.

Result: (1, Aben, 9876543210) and (1, Aben, 9123456780), so each phone gets its own row and each cell holds exactly one value.

Every cell is now atomic, so the table satisfies 1NF.

The cost is some repeated data (rollno and name appear twice), which higher normal forms handle with keys.

1NF is the entry ticket: a relation must be in 1NF before 2NF, 3NF or BCNF can even be checked.

12 **A table stores many hobbies in one column. Why does it violate 1NF? Fix it.**

IMPORTANT

It violates 1NF because a single cell of the hobbies column holds many values at once, so the values are not atomic.

The practical problem: 'find all students who like chess' forces substring searching inside cells instead of a simple WHERE hobby = 'chess'.

Updates are risky too: removing one hobby means editing a packed string, which can corrupt the other hobbies in the same cell.

The column also cannot be indexed or used in a foreign key properly.

Fix: store one hobby per row. Replace one row (1, Aben, 'chess, music') with two rows: (1, Aben, chess) and (1, Aben, music).

Now every cell holds a single value, so the table is in 1NF.

The repeating group is gone, and hobbies become queryable with ordinary SQL.

This is the standard 1NF repair: when one cell holds many values, spread them across rows.

TOPIC 6

Second Normal Form (2NF)

6

2NF: the table is in 1NF and has no partial dependency.

Partial dependency: a non-key attribute depending on only part of a composite key.

Example: key (rollno, subject); marks depends on both, but student name depends only on rollno.

Fix: split into Student(rollno, name) and Marks(rollno, subject, marks).

QUESTION BANK

13 **Explain 2NF with an example. What is partial dependency? (Model paper, Mar 2021)**

REPEATED

Second Normal Form requires: the relation is in 1NF, and it has no partial dependency.

Partial dependency: a non-prime attribute depends on only part of a composite key instead of the whole key.

Worked example: consider Marks(rollno, subject, name, marks) with composite primary key (rollno, subject).

The attribute name depends only on rollno, which is just part of the key, so it is a partial dependency; marks depends on the full key and is fine.

The anomaly: the student name repeats in every subject row, so renaming a student needs

many updates, and the name is lost if the last marks row is deleted.

Fix: decompose into Student(rollno, name) and Marks(rollno, subject, marks), keeping rollno as the link.

In Student, name depends on the whole key rollno; in Marks, marks depends on the whole key (rollno, subject).

Both tables are now in 2NF because every non-key attribute depends on the full key of its table.

Exam shortcut: 2NF only matters with a composite key; a single-column key can never have a partial dependency.

14 When does 2NF matter? What if the primary key is a single column?

IMPORTANT

2NF matters only when the primary key is composite, that is, made of two or more columns.

A partial dependency needs 'part of the key' to exist, which is impossible when the key is a single column.

Therefore a relation in 1NF with a single-column primary key is automatically in 2NF.

Example: Student(rollno, name, dept) with key rollno skips 2NF analysis entirely.

It may still fail 3NF: with rollno -> dept and dept -> hod, hod is a transitive dependency, so further decomposition is needed.

So the exam strategy is: check the key first. Composite key means test for partial dependency; single column means move straight to 3NF.

This shortcut saves time and shows the examiner you understand why 2NF exists.

In short: no composite key, no partial dependency, no 2NF problem.

TOPIC 7

Third Normal Form (3NF)

7

3NF: the table is in 2NF and has no transitive dependency.

Transitive dependency: a non-key attribute depending on another non-key attribute.

Example: rollno -> dept and dept -> hod, so hod depends transitively on rollno.

Fix: move hod into Department(dept, hod); keep Student(rollno, name, dept).

QUESTION BANK

15 Explain 3NF with an example. What is transitive dependency? (Oct 2019, Mar 2021)

REPEATED

Third Normal Form requires: the relation is in 2NF, and it has no transitive dependency.

Transitive dependency: X -> Y and Y -> Z, where Z is a non-prime attribute and Y is not a key, so Z depends on X only indirectly through Y.

Worked example: Student(rollno, name, dept, hod) with FDs rollno -> dept and dept -> hod, and rollno as the key.

hod depends transitively on rollno via dept, which violates 3NF.

The anomalies: the HOD name repeats for every student of the department; changing the HOD needs many row updates; deleting the last student of a department loses the HOD fact.

Fix: decompose into Student(rollno, name, dept) and Department(dept, hod).

Department stores each HOD exactly once with dept as key; Student keeps dept as a foreign key.

In each new table every non-key attribute depends directly on the key, so both are in 3NF.

Remember the pattern: non-key depending on non-key means transitive, and transitive means decompose.

16 A table has rollno -> city and city -> state. Is it in 3NF? Fix it.

IMPORTANT

No, it is not in 3NF. Take Student(rollno, name, city, state) with rollno as key and FDs rollno -> city and city -> state.

state depends on rollno only indirectly through city, which is a non-key attribute, so this is a transitive dependency.

3NF forbids transitive dependencies, hence the table fails 3NF.

The cost: the state of a city repeats in every student row from that city, causing update anomalies if a state name ever changes.

Fix: decompose into Student(rollno, name, city) and CityState(city, state).

CityState stores each city-state pair exactly once, with city as its primary key.

Student keeps city as a foreign key linking the two tables.

In both tables every non-key attribute depends directly on the key, so both are in 3NF.

TOPIC 8

8

Normal Forms: Overview

Normalization removes redundancy step by step: 1NF removes repeating groups, 2NF removes partial dependency, 3NF removes transitive dependency, BCNF tightens the rule to candidate keys.

The classic exam essay asks all normal forms together with examples.

QUESTION BANK

17 **Explain various types of normal forms with examples. (Oct 2019, Mar 2021)**

SURE MARK

1NF: no multi-valued or repeating columns; every cell holds one atomic value. Example: split Hobbies('cricket, chess') into separate rows.

2NF: in 1NF, plus no partial dependency - every non-key column must depend on the WHOLE key. Example: Marks(rollno, subject, student_name, mark) has student_name depending only on rollno, so split into Student(rollno, student_name) and Marks(rollno, subject, mark).

3NF: in 2NF, plus no transitive dependency - non-key columns must depend only on the key. Example: Emp(empno, deptno, deptname) has deptname depending on deptno, so split into Emp(empno, deptno) and Dept(deptno, deptname).

BCNF: stricter than 3NF - every determinant must be a candidate key. Example: Teaching(student, subject, teacher), where each teacher teaches one subject, can be in 3NF yet violate BCNF; split into Teaches(teacher, subject) and Enrolled(student, teacher).

Higher forms (4NF, 5NF) handle multi-valued and join dependencies and are rarely asked.

This exact essay repeated verbatim in the Oct 2019 and Mar 2021 Part C - prepare one clean version covering all four.

18 **What is the purpose of normalization in database design? (Model paper)**

IMPORTANT

Normalization organises tables to remove redundancy and the anomalies it causes.

Its purpose is that every fact is stored exactly once, in the right table.

It breaks big messy tables into smaller ones using functional dependencies, from 1NF up to BCNF.

Benefits: no update, insertion or deletion anomalies, less storage waste, and consistent data.

Example: splitting Enroll(rollno, name, course) into Student(rollno, name) and Enroll(rollno, course).

In short, normalization trades a little join effort for a database that stays correct.

TOPIC 9

9

BCNF

BCNF (Boyce-Codd Normal Form): every determinant must be a candidate key.

A determinant is any attribute set on the left side of a FD.

BCNF is stricter than 3NF: a table can be in 3NF but not in BCNF.

Example: in Teaching(student, subject, teacher) with teacher -> subject, teacher is not a key, so it violates BCNF.

QUESTION BANK

19 **What is BCNF? How is it stricter than 3NF? (Dec 2021)**

IMPORTANT

BCNF (Boyce-Codd Normal Form) rule: for every non-trivial FD X -> Y in the relation, X must be a candidate key.

In plain words: every determinant, every left side of a FD, must be a key.

3NF is looser: it permits a non-key determinant X -> Y provided Y is a prime attribute, that is, part of some candidate key.

BCNF closes that loophole; even when Y is prime, X must still be a key.

Worked example: Teaching(student, subject, teacher) with FDs {student, subject} -> teacher and teacher -> subject.

teacher -> subject passes 3NF because subject is prime, but it violates BCNF because teacher is not a candidate key.

Fix: decompose into TeacherSubject(teacher, subject) and StudentTeacher(student, teacher).

Every relation in BCNF is automatically in 3NF, but the reverse is false; BCNF is the strongest normal form based on FDs.

In the exam, always list the FDs, find the candidate keys, then test each FD's left side: non-key left side means not BCNF.

20 **Give an example of a table in 3NF but not in BCNF.**

IMPORTANT

Consider Teaching(student, subject, teacher) with FDs {student, subject} -> teacher and teacher -> subject.

Assume each teacher teaches only one subject, but a subject may be taught by several teachers.

Candidate keys: {student, subject} is a key, and {student, teacher} is also a key, because teacher determines subject.

Check 3NF: for teacher -> subject, the right side (subject) is a prime attribute, part of a candidate key, so 3NF permits this FD. The table is in 3NF.

Check BCNF: for teacher -> subject, the left side (teacher) is not a candidate key, and BCNF demands every determinant be a key. So the table is not in BCNF.

The anomaly: a teacher's subject repeats across rows, and deleting a student can destroy the fact that a teacher teaches a subject.

Fix: decompose into TeacherSubject(teacher, subject) and StudentTeacher(student, teacher).

In TeacherSubject, teacher is the key; in StudentTeacher, {student, teacher} is the key.

Both new relations satisfy BCNF, since every determinant is now a key.

TOPIC 10

10

Denormalization

Denormalization deliberately adds redundancy back into normalised tables.

It speeds up read-heavy queries by avoiding many joins.

Used in data warehouses and reporting systems.

Trade-off: faster reads, but slower writes and more storage.

QUESTION BANK

21 What is denormalization? When is it used?

IMPORTANT

Denormalization is the deliberate re-introduction of controlled redundancy into a normalized database to improve read performance.

Normalization splits tables to remove duplication; denormalization merges some of that data back, typically to avoid expensive joins.

Example: a reporting table may store the customer name inside every order row, so a sales report needs no join with the Customer table.

It is used mainly in data warehouses and reporting systems, where queries read huge volumes of data but writes are rare and batched.

It is also used on live systems when a specific join-heavy query is a proven bottleneck.

The trade-off: reads get faster, but storage grows and every write must update all copies consistently.

It is therefore applied selectively, only to the hot queries that truly need the speedup.

Denormalization is a performance decision, not a design failure; it is chosen consciously after normalization is understood.

22 What problems can denormalization create?

IMPORTANT

The biggest problem is update anomalies: the same fact stored in two places can drift out of sync when one copy is updated and the other is missed.

Example: if the customer name is copied into every order row, renaming the customer needs updates across thousands of rows, and a partial update leaves contradictory names.

Storage cost rises because data is deliberately duplicated.

Write performance drops: every insert or update must touch all copies, usually inside one transaction.

Consistency guarding moves from the schema to the application, which is error-prone.

Delete anomalies can appear too: removing the last copy of a fact in one table may orphan the copies elsewhere.

These costs are accepted only where read speed matters far more than write speed and storage, such as data warehouses.

So the exam answer is: faster reads, but anomalies, extra storage, slower writes and harder maintenance.

TOPIC 11

11

ACID Properties

Atomicity: a transaction is all or nothing; no half-done updates.

Consistency: every transaction moves the database from one valid state to another.

Isolation: concurrent transactions behave as if run one after another.

Durability: once committed, data survives crashes and power failure.

QUESTION BANK

23 **Explain the ACID properties with examples.**

IMPORTANT

ACID is the set of four guarantees a DBMS gives every transaction: Atomicity, Consistency, Isolation and Durability.

Atomicity: a transaction is all or nothing. In a fund transfer, the debit and the credit either both happen or both are undone; a crash midway cannot leave money debited but not credited.

Consistency: every transaction moves the database from one valid state to another, preserving all rules. After the transfer, the sum of all balances is unchanged and no account goes negative.

Isolation: concurrent transactions behave as if they ran one after another. Two transfers on the same account do not interleave and corrupt the balance.

Durability: once a transaction commits, its effects survive crashes and power failure. After the transfer commits, the new balances stay even if the server dies a second later.

Together these four properties are why databases are trusted with money, bookings and other critical data.

If any one of them fails, the database can hold wrong or half-written data, which is why the DBMS enforces all four for every transaction.

Memory hook: A for all-or-nothing, C for correct states, I for isolated execution, D for data that survives crashes.

24 **What happens if atomicity is violated during a bank transfer?**

IMPORTANT

Suppose a transfer debits Rs 5000 from account A, and then the system crashes before crediting account B.

Without atomicity, the debit stays but the credit never happens, so Rs 5000 simply vanishes from the database.

Consistency breaks too: the total money recorded by the bank no longer matches reality.

Customers see wrong balances, statements disagree with the bank's books, and trust collapses.

The DBMS prevents this by treating the debit and credit as one atomic unit: on any failure it rolls back the debit as well.

Rollback restores the pre-transfer state, so the transfer either completes fully or has no effect at all.

That is the point of atomicity: it turns a pair of updates into a safe, all-or-nothing transaction.

Partial work is the most dangerous kind of wrong data: it looks complete but is not.

TOPIC 12

12

Transactions, Schedules and Concurrency Problems

A transaction is a set of operations treated as one unit, e.g. a fund transfer.

A schedule is the order in which operations of concurrent transactions run.

Lost update: two transactions overwrite each other and one update is lost.

Dirty read: reading data written by an uncommitted transaction that later rolls back.

Unrepeatable read: reading the same row twice gives different values.

QUESTION BANK

25 Briefly describe the states of a transaction. (Oct 2019, Mar 2021)

REPEATED

Active: the transaction is running and executing its reads and writes.

Partially committed: the last statement finished; the DBMS now checks whether everything can be made permanent.

Committed: all changes are safely recorded - the transaction succeeded.

Failed: something went wrong (error or crash) and normal execution cannot continue.

Aborted: after failure, the transaction is rolled back and the database is restored to its earlier state.

A transaction moves Active -> Partially committed -> Committed on success, or Active/Partially committed -> Failed -> Aborted on failure.

26 What is a transaction? Explain three problems of concurrent execution.

IMPORTANT

A transaction is a logical unit of work: a sequence of database operations that must either complete fully or have no effect at all. Example: a fund transfer is a debit plus a credit as one transaction.

Problem 1, lost update: T1 and T2 both read balance 1000, each adds 100, and each writes back its result. The final balance is 1100 instead of 1200 because the second write overwrites the first; one update is lost.

Problem 2, dirty read: T1 reads a balance that T2 has written but not yet committed. If T2 then rolls back, T1 has acted on data that never officially existed.

Problem 3, unrepeatable read: T1 reads a row, T2 changes and commits it, then T1 reads the same row again and gets a different value, so T1 cannot trust its own earlier read.

All three arise only when transactions run concurrently without control.

They are exactly what concurrency control prevents: locks, two-phase locking and serializability rules exist to rule out these failures.

In the exam, always give the T1/T2 story for each problem; the concrete example is what earns full marks.

Remember the trio as: lost update (overwrite), dirty read (uncommitted), unrepeatable read (changed between reads).

27 What is a schedule? What makes a schedule correct?

IMPORTANT

A schedule is the chronological order in which the operations of concurrently running transactions are interleaved by the DBMS.

Example: a schedule of T1 and T2 could be read1(A), read2(A), write1(A), write2(A), commit1, commit2.

A schedule is correct mainly if it is serializable: its final effect equals the effect of running the same transactions one after another in some serial order.

It should also be recoverable: no transaction commits based on uncommitted data of another, so crash recovery stays possible.

The DBMS scheduler enforces these properties using locking protocols like 2PL or timestamp ordering.

In short: a schedule is correct when concurrency changed the execution order but not the outcome.

Serial schedules (no interleaving at all) are always correct but slow; serializable schedules give the same correctness with far better performance.

That trade-off is the whole reason concurrency control theory exists.

TOPIC 13

Lock-Based Concurrency Control

13

Locks stop two transactions from changing the same data at the same time.

Shared lock (S): many transactions can read together, but nobody can write.

Exclusive lock (X): only one transaction can write; nobody else can read or write.

A transaction must release its locks when done so others can proceed.

QUESTION BANK

28 Explain shared and exclusive locks with an example.

IMPORTANT

Locks are how the DBMS stops two transactions from changing the same data at the same time; a transaction must hold a lock on a data item before touching it.

Shared lock (S): taken for reading. Many transactions can hold shared locks on the same item together, because reading does not change anything.

Exclusive lock (X): taken for writing. Only one transaction can hold it, and while it is held nobody else may read or write the item.

Compatibility rule: S is compatible with S, but X conflicts with both S and X, so a writer waits for all readers and other writers.

Example: T1 takes a shared lock on the balance row to read it; T2 wants to update the balance and requests an exclusive lock, so T2 waits until T1 releases.

Rule of thumb: take a shared lock before reading, an exclusive lock before writing, and release locks when the transaction finishes.

This discipline is what prevents lost updates and dirty reads.

Without locks, concurrent transactions would freely overwrite each other's work.

29 What happens if a transaction reads data without taking any lock?

IMPORTANT

It can suffer a dirty read: it may see a value written by another transaction that later rolls back, so the data it acted on never officially existed.

It can suffer an unrepeatable read: reading the same row twice may return different values, because a concurrent transaction changed it in between.

It can read a half-updated state: with no lock, nothing stops a writer from modifying the row in the middle of the read.

Its own later writes can then be based on stale values, producing lost updates that corrupt totals.

Locks exist precisely to rule out these failures: every read takes a shared lock and every write takes an exclusive lock.

Reading without a lock trades a little speed for completely unreliable results.

A DBMS must never allow that trade-off, because wrong data silently spreads into every later computation.

So the rule is absolute: no lock, no touch.

30 What affects the performance of locking?

IMPORTANT

Lock granularity: row-level locks let many transactions work on different rows of the same table at once, while table-level locks block the whole table and destroy concurrency.

Lock overhead: acquiring, checking and releasing locks costs CPU time and memory, so heavy locking slows even simple operations.

Blocking and waiting: a transaction stuck waiting for a lock held by another delays its whole chain, and long waits cascade into deadlocks.

Lock duration: holding locks until commit, as in strict 2PL, is safest but blocks others longest; releasing early gains speed at the cost of safety.

Lock escalation: when a transaction holds too many fine locks, the system may merge them into one coarse lock, suddenly reducing concurrency.

Deadlock handling itself costs performance: detection scans the wait-for graph, and victim rollback wastes completed work.

Good design balances the two sides: locks fine-grained and short-lived enough for speed, strict enough to keep data correct.

In the exam, granularity, overhead and waiting time are the three points that must appear.

TOPIC 14

Two-Phase Locking, Serializability, Recoverability

14

2PL: a transaction has a growing phase (takes locks, releases none) then a shrinking phase (releases locks, takes none).

2PL guarantees serializability: the result matches some serial order.

Serializability means concurrent execution is as correct as running transactions one by one.

Recoverable schedule: a transaction commits only after the transactions it read from have committed.

Cascadeless schedules avoid cascading rollbacks.

QUESTION BANK

31 What is Two-Phase Locking? Explain its two phases.

IMPORTANT

Two-Phase Locking (2PL) is a protocol that guarantees serializable schedules by splitting a transaction's locking behaviour into two strict phases.

Phase 1, growing: the transaction may acquire locks but may not release any lock.

Phase 2, shrinking: the transaction may release locks but may not acquire any new lock.

The lock point, the moment the first lock is released, separates the two phases; after it, no new locks are allowed.

Example: T1 locks A, locks B, updates both, then releases A and B. All lock requests happened before any release, so 2PL holds.

Counter-example: T1 locks A, releases A, then locks B. This violates 2PL because a lock is acquired after a release.

2PL guarantees conflict serializability: any schedule of 2PL transactions has the same effect as some serial order.

Limitation: plain 2PL does not prevent cascading rollbacks. Strict 2PL, which holds all exclusive locks until commit, fixes this and is what real systems use.

So 2PL gives correctness, and strict 2PL adds recoverability on top.

32 What is serializability? Why does it matter?

IMPORTANT

A schedule is serializable if its final effect on the database is identical to the effect of running the same transactions one after another in some serial order.

Example: if an interleaved schedule of T1 and T2 leaves the same balances as running T1 fully and then T2 fully, the schedule is serializable.

It matters because it is the correctness standard for concurrency: it proves that interleaving operations did not corrupt anything.

Without serializability, lost updates, dirty reads and inconsistent totals can slip through silently.

2PL is the standard practical way to guarantee serializability.

Serial schedules are trivially serializable but slow; serializable schedules give the same safety with far better performance.

Conflict serializability, tested with a precedence graph, is the version the DBMS actually checks.

In one line: serializability means concurrency changed the order but not the outcome.

33 What is a recoverable schedule?

IMPORTANT

A schedule is recoverable if no transaction commits before the transactions whose uncommitted data it read have committed.

Example: if T1 reads a value written by T2, then T1 must commit only after T2 commits; letting T1 commit first makes the schedule unrecoverable.

Why it matters: if T2 rolls back after T1 has already committed, the DBMS cannot undo T1's committed work, leaving the database permanently inconsistent.

A stronger form is cascadelessness: a transaction reads only committed data, so one rollback never forces a chain of other rollbacks.

Strict 2PL, which holds all exclusive locks until commit, automatically produces cascadeless and therefore recoverable schedules.

Recoverability is what makes crash recovery possible: the log can safely redo and undo because the commit order respected read dependencies.

Without it, a crash could leave committed transactions depending on rolled-back data, which no recovery procedure can repair.

So recoverability is the rule that keeps the commit order honest.

34 Differentiate between conflict serializability and view serializability. (Model paper)

IMPORTANT

Both test whether a concurrent schedule behaves like some serial (one-after-another) order.

Conflict serializability: a schedule is safe if its conflicting operations (read-write or write-write on the same item) can be reordered into a serial order; tested with a precedence graph - no cycles means safe.

View serializability: a weaker test - the schedule only needs to give each read the same value, and the same final writes, as some serial schedule.

Every conflict-serializable schedule is view-serializable, but not vice versa.

In practice DBMSs enforce conflict serializability (via 2PL) because it is easy to test; view serializability is harder to check.

Remember the one-line difference: conflict looks at operation order, view looks at what each read actually saw.

TOPIC 15

Deadlocks

15

Deadlock: two transactions each wait for a lock held by the other, forever.

Example: T1 locks A and wants B; T2 locks B and wants A.

Prevention: order all lock requests the same way so cycles cannot form.

Detection: build a wait-for graph; a cycle means deadlock, so roll back one transaction.

Timeout: if a transaction waits too long, roll it back and retry.

QUESTION BANK

35 What is a deadlock? How is it handled in DBMS?

IMPORTANT

A deadlock is a circular wait between transactions: each transaction holds a lock that another one needs, so none of them can ever proceed.

Example: T1 locks row A and requests row B; T2 locks row B and requests row A. Each waits for the other forever.

Handling method 1, prevention: impose a global order on lock requests, always lock A before B, so waiting cycles can never form.

Handling method 2, detection and recovery: the DBMS builds a wait-for graph and checks it periodically; on finding a cycle it picks a victim transaction and rolls it back, freeing its locks.

Handling method 3, timeouts: if a transaction waits longer than a fixed limit, the system aborts it and retries, breaking the wait cheaply.

Prevention is simple but restrictive; timeouts are simple but may abort transactions that were not really deadlocked.

Most real DBMSs use detection with victim selection, since it allows maximum concurrency and only pays the cost when a deadlock truly occurs.

The victim is usually the transaction with the least work done or the lowest priority, so the wasted work is minimal.

36 How does the wait-for graph detect a deadlock?

IMPORTANT

The wait-for graph has one node for each active transaction in the system.

A directed edge T1 -> T2 is drawn whenever T1 is waiting for a lock currently held by T2.

The DBMS updates this graph as transactions request and release locks, and checks it periodically for cycles.

A cycle means a deadlock: following the edges loops back to the start, so every transaction in the cycle waits on another one in the cycle and nobody can proceed.

Example: edges T1 -> T2 and T2 -> T1 form a cycle, proving T1 and T2 are deadlocked over the locks they hold.

Recovery: the system chooses a victim, usually the transaction with the least completed work or lowest priority, rolls it back and releases its locks, which breaks the cycle.

The surviving transactions then continue normally, and the victim is typically retried.

The graph turns an invisible waiting problem into a visible cycle that the DBMS can detect and fix automatically.

TOPIC 16

Transaction Support in SQL and Crash Recovery

16

COMMIT: saves all changes of the transaction permanently.

ROLLBACK: undoes all changes since the last commit.

SAVEPOINT: marks a point inside a transaction to roll back to partially.

Crash recovery replays the log: redo committed transactions, undo uncommitted ones.

The log records every change before it is applied, so nothing is lost.

QUESTION BANK

37 Explain COMMIT, ROLLBACK and SAVEPOINT with examples. (Model paper)

IMPORTANT

COMMIT ends a transaction successfully and makes all its changes permanent and durable.

ROLLBACK ends a transaction by undoing every change made since the last commit, restoring the old state.

SAVEPOINT marks a named point inside a transaction, so a later rollback can stop partway instead of undoing everything.

Example of all three together:

```
UPDATE Account SET balance = balance - 500 WHERE id = 1;
SAVEPOINT s1;
UPDATE Account SET balance = balance + 500 WHERE id = 2;
ROLLBACK TO s1;
COMMIT;
```

Here the debit of account 1 stays, but the credit to account 2 is undone by rolling back to s1, and COMMIT then saves the debit permanently.

Savepoints are useful when one step of a multi-step transaction fails but the earlier steps are still valid and worth keeping.

38 How does a DBMS recover from a crash?

IMPORTANT

The DBMS keeps a write-ahead log: every change is recorded in the log on disk before it is applied to the database itself.

When the system restarts after a crash, the recovery manager scans this log from the last checkpoint.

Committed transactions are redone: their logged changes are reapplied so their effects survive the crash, which gives durability.

Uncommitted transactions are undone: their partial changes are rolled back so no half-done work remains, which gives atomicity.

Checkpoints, periodic snapshots of the database state written to the log, shorten the scan by marking how far back recovery must look.

Because of the write-ahead rule, a crash can never leave the database in a half-updated state.

Everything is either fully applied or fully removed, so the database always restarts in a consistent state.

This is why the log is the most critical structure in a DBMS: it is the difference between a crash

and a catastrophe.

39 Explain the types of failures in a DBMS. (Dec 2021)

IMPORTANT

Transaction failure: a single transaction aborts due to an error (bad input, deadlock); only its changes are rolled back.

System crash: power loss or OS failure wipes memory; committed data on disk survives, uncommitted work is undone using the log.

Disk failure: the storage itself is damaged; recovery needs backups plus the log replayed from the last backup.

The DBMS handles the first two with write-ahead logging (undo/redo); disk failure needs archived backups.

Knowing the failure types decides the recovery strategy, which is why the exam asks it before crash recovery.

Failure types decide the recovery method, so this question pairs naturally with crash recovery in the same exam.